



K.R. Mangalam University

School of Engineering and Technology

Task -3 Generate visualizations and comparative charts

Design & Analysis of Algorithms

Course Code - ENCA351

(2026-27)

Submitted by:

Diksha Goel
2401201173
BCA (AI & DS) - A

Submitted To:

Dr. Aarti Sangwan
Faculty, SOET

A. Search algorithms

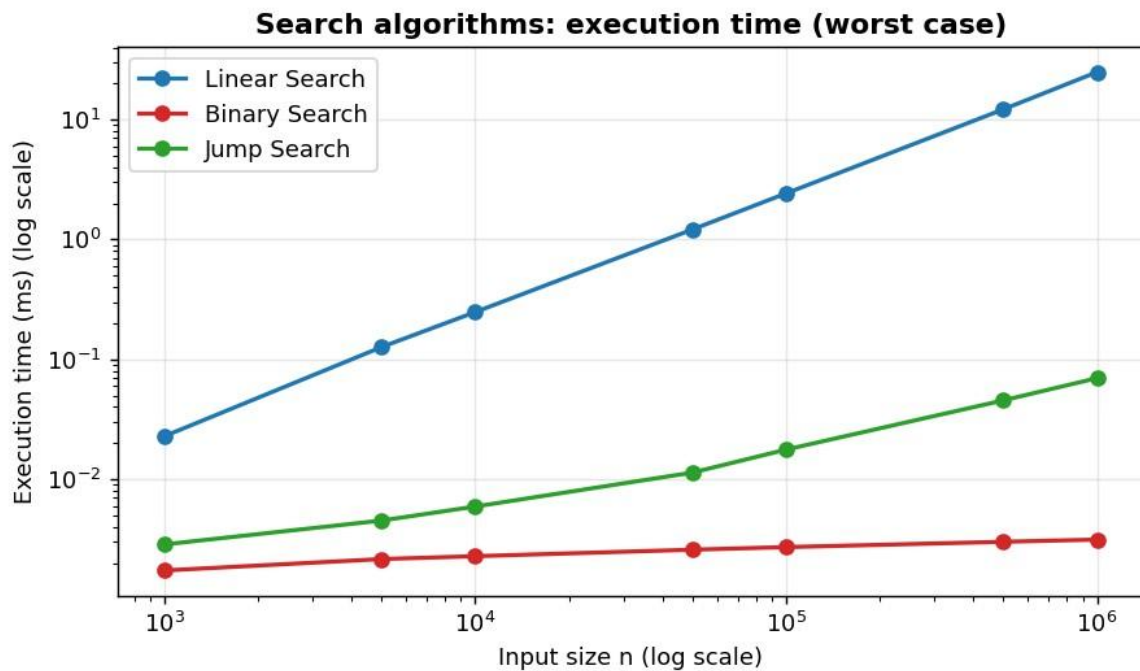


Figure 2: Execution time of linear, binary and jump search (worst case).

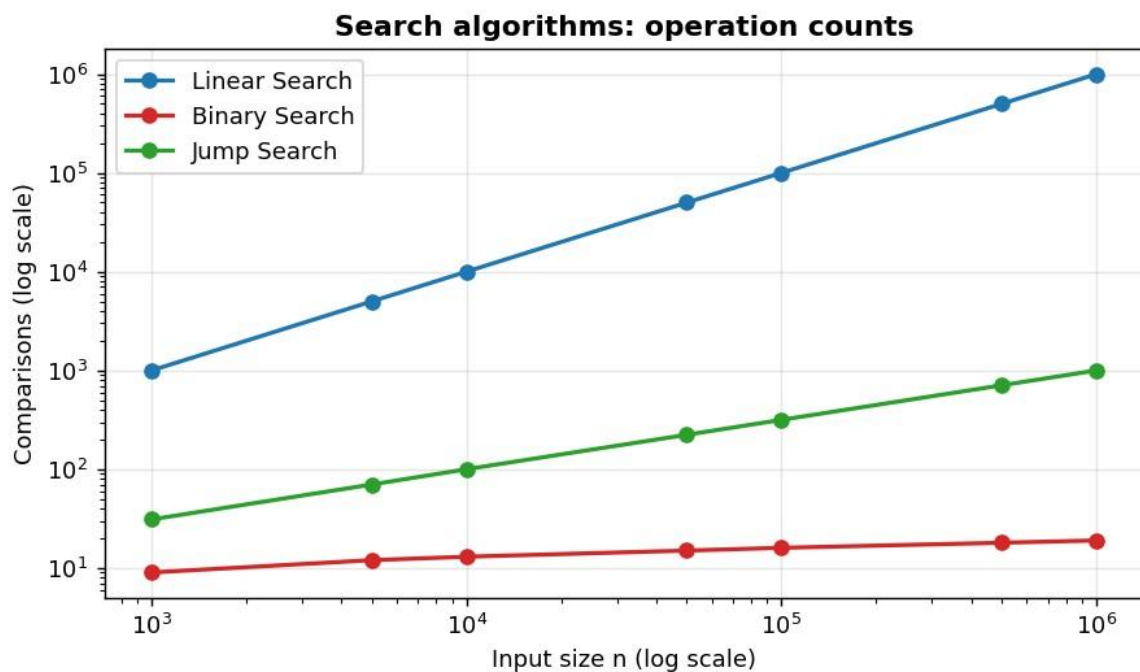


Figure 3: Number of comparisons performed by each search algorithm.

Performance observations

- **Linear search scales linearly.** Comparisons grew exactly with n (1,000,000 comparisons for $n = 1,000,000$) and the time slope was 1.01. At $n = 1,000,000$ it took 24.911 ms.

- **Binary search is almost flat.** It needed only 19 comparisons for one million elements ($\approx \log_2 n$) and 0.0031 ms, about 7,942× faster than linear search at that size.
- **Jump search sits in between.** 1,000 comparisons ($= \sqrt{n}$) and 0.070 ms, with a measured slope of 0.47, which matches the theoretical 0.5.
- **Memory is constant for all three.** Each uses only a few index variables (peak ≈ 0.2 KB), confirming $O(1)$ auxiliary space.
- **Trade-off.** Binary and jump search need sorted data; for a one-off search of unsorted data, linear search avoids the cost of sorting.

B. Factorial implementations

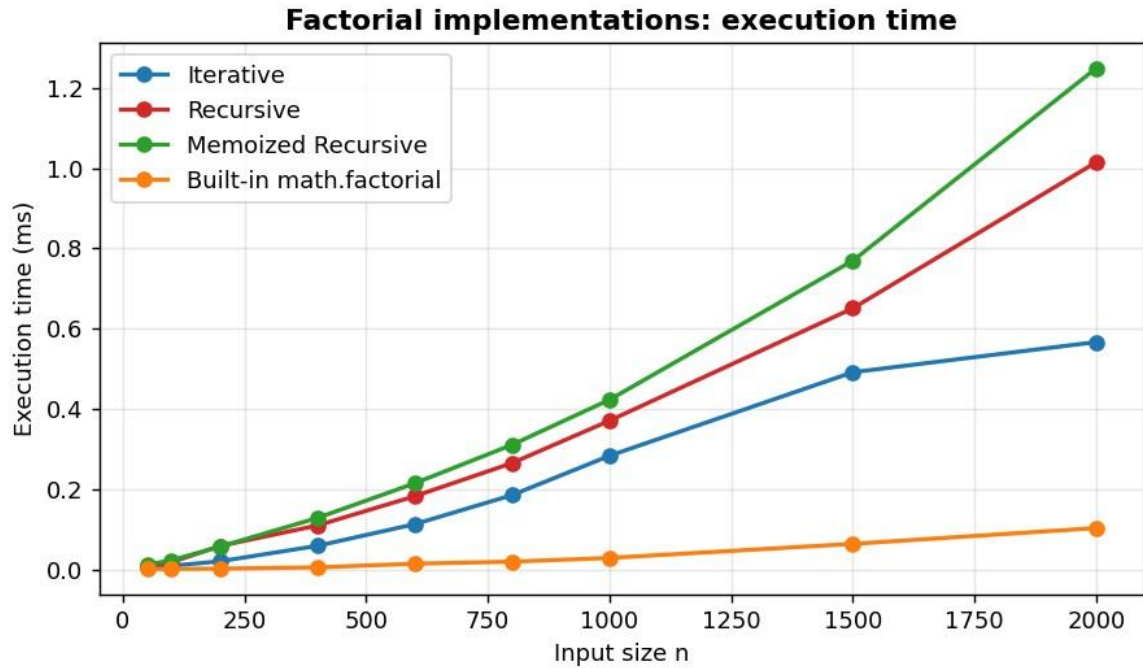


Figure 4: Execution time of the four factorial implementations.

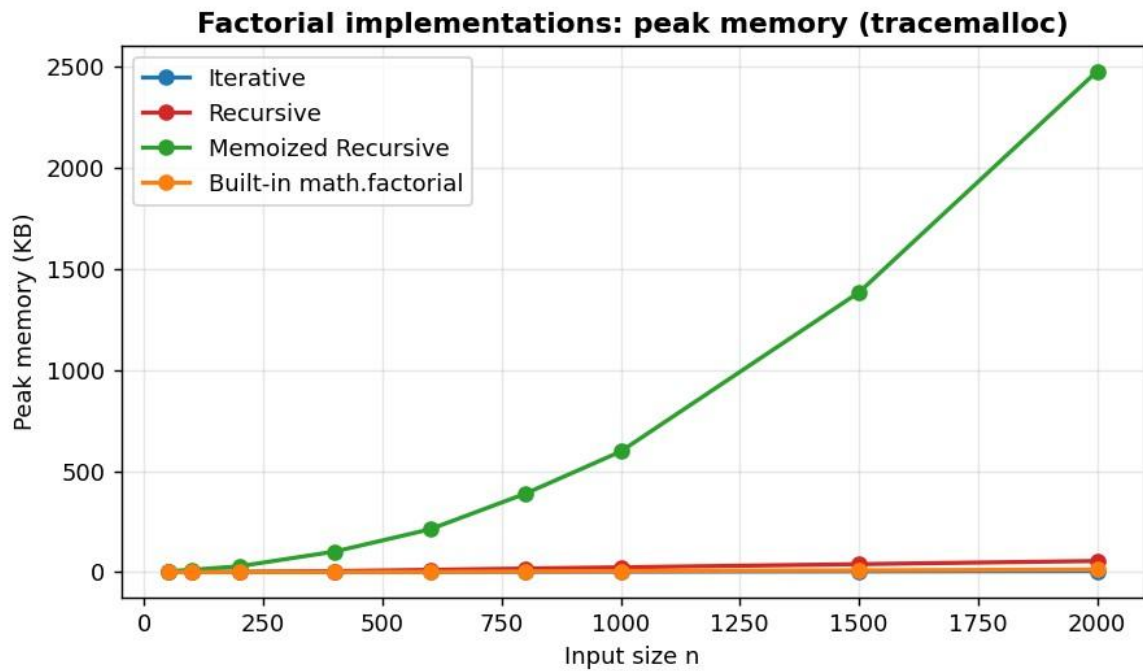


Figure 5: Peak memory allocated by each factorial implementation.

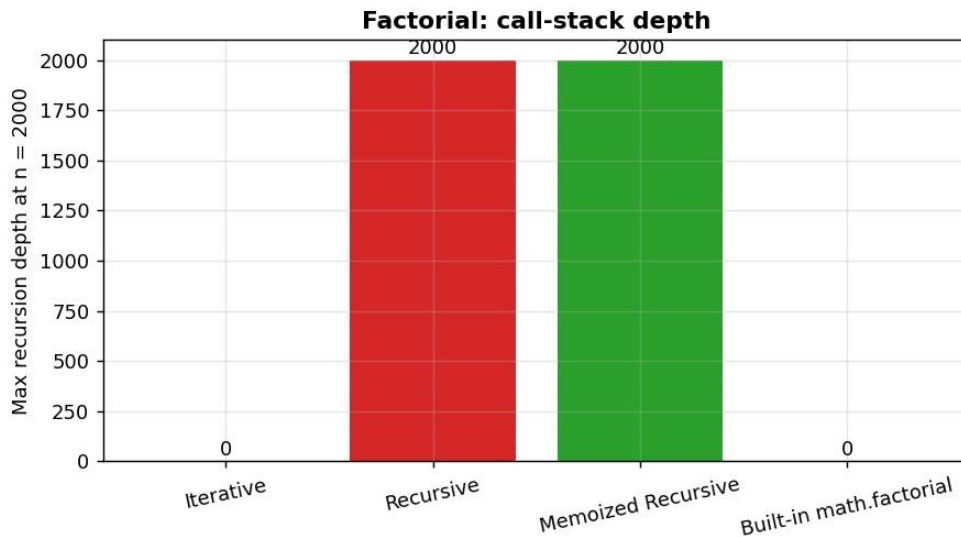


Figure 6: Maximum call-stack depth at $n = 2000$.

Performance observations

- **Same operation count, different cost.** Iterative, recursive and memoized versions all perform exactly $n - 1$ multiplications, yet at $n = 2000$ they took 0.567 ms, 1.015 ms and 1.249 ms respectively. Function-call overhead makes recursion slower than a loop.
- **Built-in is fastest.** `math.factorial` took 0.103 ms at $n = 2000$, about 5.5× faster than the iterative loop, because the whole loop runs in optimised C code.
- **Time is slightly super-linear.** Observed slopes were 1.41 (iterative), 1.28 (recursive) and 1.27 (memoized) rather than 1.0: factorial values become huge integers, and multiplying big integers costs more than multiplying small ones.
- **Space differs the most.** Recursion reaches a call-stack depth of 2,000 at $n = 2000$ while the iterative version needs none ($O(n)$ vs $O(1)$ stack). The memoized version stored every intermediate factorial and used 2.42 MB at $n = 2000$; doubling n from 1000 to 2000 multiplied its memory by 4.2, i.e. roughly quadratic growth because the cached numbers are themselves growing.
- **Memoization did not help here.** Each top-level call starts with an empty cache and every sub-problem is visited once, so memoization only adds overhead. It pays off only when the same function is called repeatedly, e.g. computing many factorials in sequence.

C. Loop-based programs

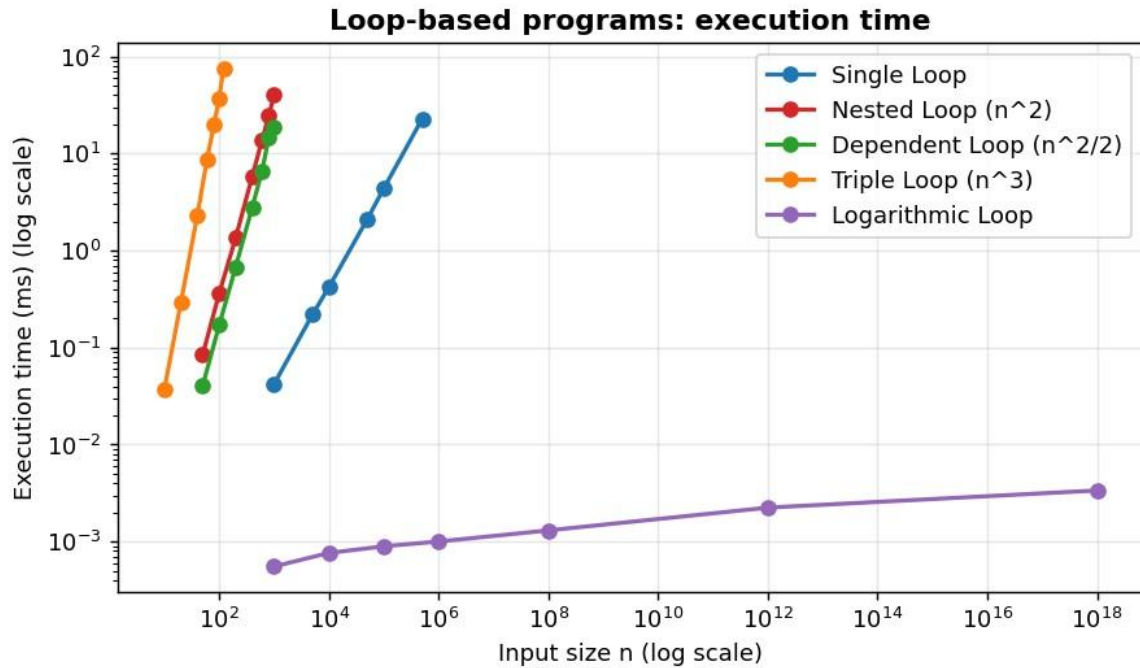


Figure 7: Execution time of loop-based programs (log-log scale).

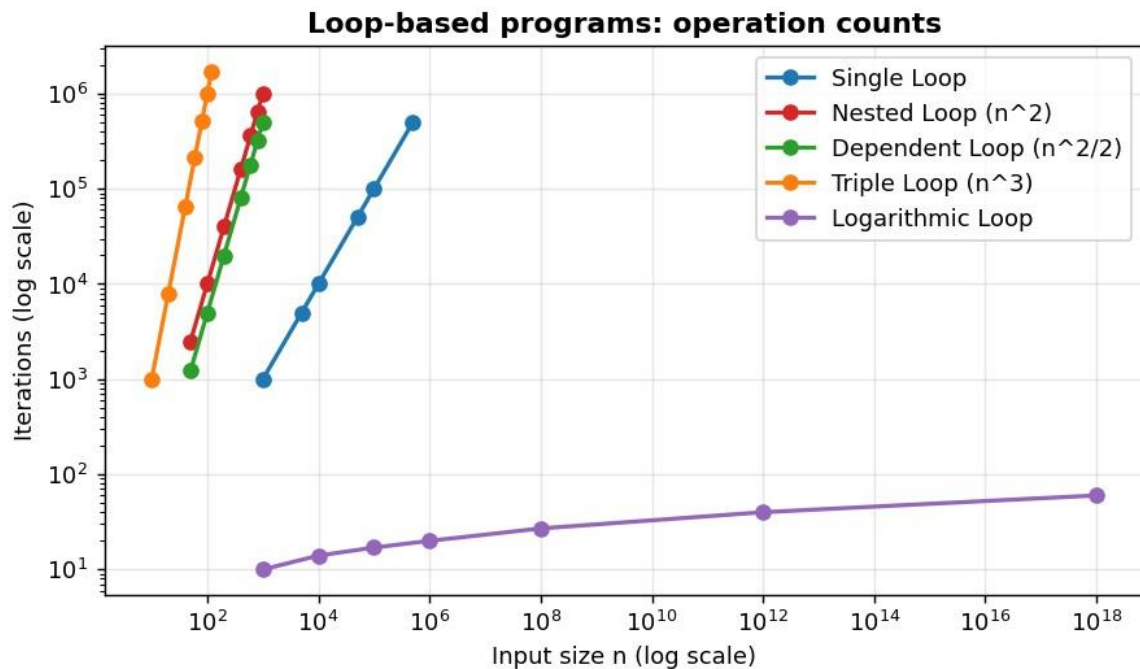


Figure 8: Iteration counts of loop-based programs.

Performance observations

- **Operation counts match theory exactly.** Single loop = n , nested = n^2 , dependent = $n(n-1)/2$ and triple = n^3 iterations (ops/f(n) = 1.000 for all). The logarithmic loop ran only 60 iterations even for $n = 10^{18}$.
- **Time follows the same exponents.** Observed log-log slopes were 1.01, 2.05, 2.07, 3.05 and 0.05 for single, nested, dependent, triple and logarithmic loops, against theoretical 1, 2, 2, 3 and ≈ 0 .

- **Constant factors matter.** The dependent loop does half the iterations of the nested loop at $n = 1000$ (499,500 vs 1,000,000) and ran in 19.0 ms vs 40.4 ms. Both are $O(n^2)$, but the constant differs.
- **Practical impact.** The triple loop at only $n = 120$ (1.7 million iterations) took 76.2 ms, longer than a single loop of 500,000 iterations (22.8 ms), showing how fast polynomial growth overtakes linear growth.
- **Memory is constant.** All loop programs use $O(1)$ extra space (peak < 0.4 KB).

D. Overall comparison

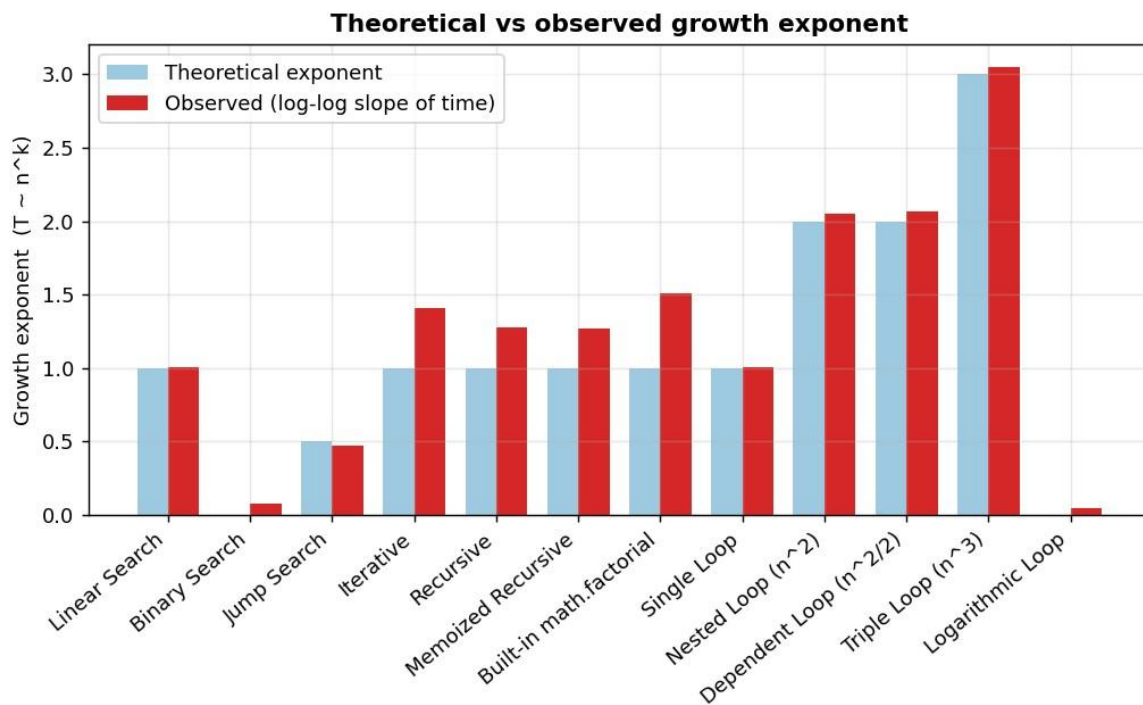


Figure 9: Theoretical growth exponent vs observed log-log slope for all 12 algorithms.

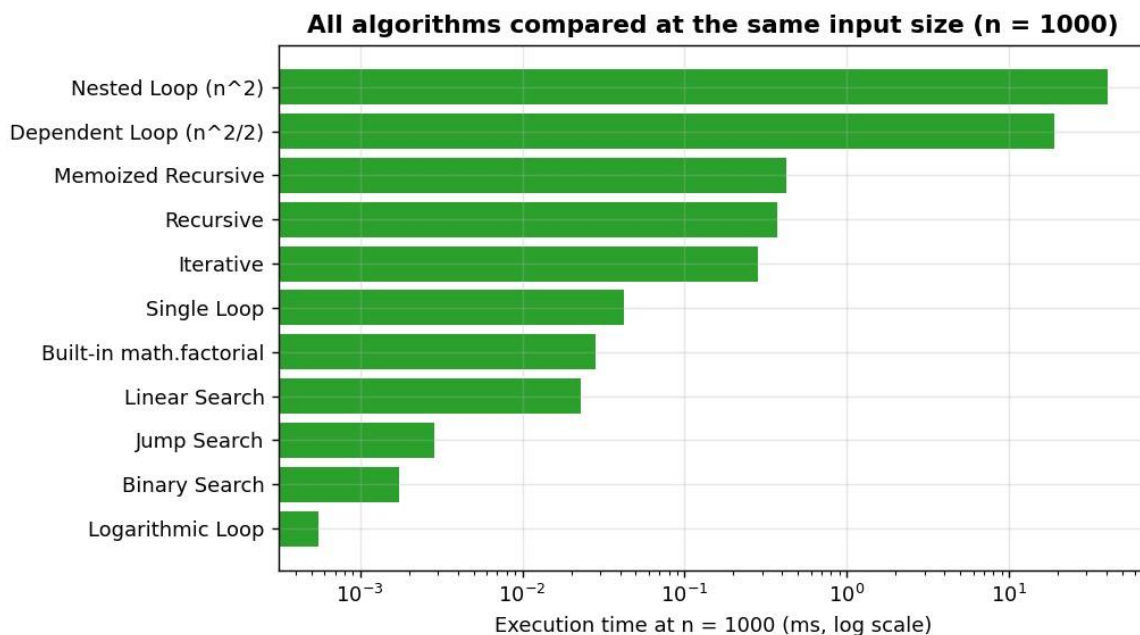


Figure 10: All algorithms measured at the same input size ($n = 1000$).

Overall observations

- Observed exponents track the theoretical ones closely; the only systematic deviation is the factorial group, caused by big-integer arithmetic.
- At the same n , algorithm choice changes running time by several orders of magnitude (compare binary search and the nested loop in Figure 10).

- Choosing a better algorithm (binary vs linear search, built-in vs hand-written factorial) gives bigger gains than any constant-factor tuning.